Adaptive Network Security Policies via Belief Aggregation and Rollout

Kim Hammar, Yuchao Li, Tansu Alpcan, *Senior Member, IEEE*, Emil C. Lupu, *Member, IEEE*, and Dimitri P. Bertsekas

***Abstract*—Evolving security vulnerabilities and shifting operational conditions require frequent updates to network security policies. These updates include adjustments to incident response procedures and modifications to access controls, among others. Reinforcement learning methods have been proposed for automating such policy adaptations, but most methods in the research literature lack performance guarantees and adapt slowly to changes. In this paper, we address these limitations and present a method for computing security policies that is scalable, offers theoretical guarantees, and adapts quickly to changes. The method uses a model or simulator of the system, which is updated when changes occur, and combines three components: belief estimation through particle filtering, offline policy computation through feature-based aggregation, and online policy adaptation through rollout. In particular, feature-based aggregation enables scalable offline optimization of a policy, while rollout adapts the policy online to changes in the system model without repeating the offline optimization. We analyze the approximation error of the aggregation and show that the rollout efficiently adapts policies to changes under certain conditions. Simulations and testbed results demonstrate that our method outperforms state-of-the-art methods on several benchmarks, including CAGE-2.**

***Index Terms*—Cybersecurity, aggregation, rollout, decision theory, dynamic programming, POMDP, reinforcement learning.**

I. Introduction

NETWORK security policies dictate how security measures are implemented and applied to protect a networked system against attacks. Such policies can be enforced at the physical, network, and service layers and include access control, flow control, and intrusion response policies, among others. Traditionally, such policies have been defined, implemented, and updated by domain experts [1]. Although this approach can provide effective security policies for systems that change infrequently, it becomes impractical for systems with frequent changes, such as those caused by shifting operational requirements, fluctuating workloads, component failures, or software updates. These dynamic changes make policy adjustments and reconfigurations an inherent part of operations, necessitating adaptive approaches to managing security policies [2].

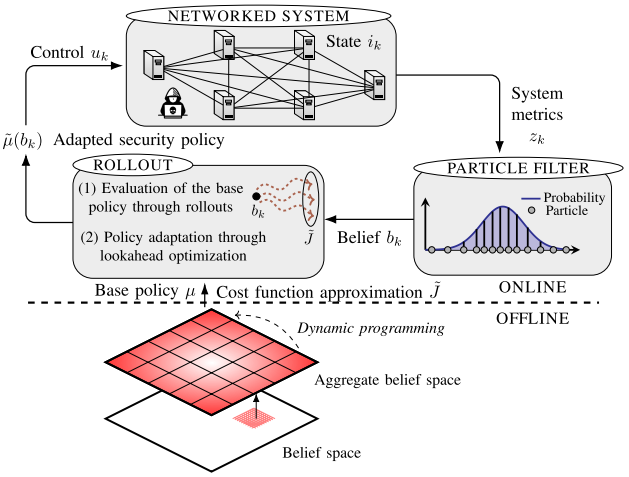


Fig. 1. Our method for computing adaptive network security policies. A base policy and cost function are computed offline via dynamic programming in an aggregate belief space, where beliefs represent uncertainty about the system's security state. At runtime, the belief is estimated via particle filtering and the base policy is adapted via rollout simulations and lookahead optimization guided by the cost function. This lookahead allows the system to anticipate possible threats and assess the impact of various security controls.

A promising solution is to frame the problem of obtaining an effective security policy as a sequential decision-making problem, which enables automatic policy adaptation via reinforcement learning (RL) [3]. In this formulation, a security policy is defined as a function that prescribes security controls (e.g., access or flow controls) based on a *belief* about the system's security state (e.g., whether it is compromised or not). This belief is defined as a probability distribution over possible system states and is updated sequentially based on system metrics (e.g., logs and alerts). These metrics are also used to track changes to the system and iteratively adapt the security policy to meet a given objective quantified by a cost function. Recent work shows the potential of this approach to adapt a broad range of security policies, including intrusion response [4], [5], penetration testing [6], deception [7], replication [8], and fuzzing [9] policies. Despite these advances, important limitations remain. In particular, most methods for policy adaptation proposed in the literature use *deep* RL, which has

Received 20 July 2025; revised 3 May 2026; accepted 25 June 2026. Date of publication 1 July 2026; date of current version 21 July 2026. This work was supported by Swedish Research Council under Contract 2024-06436. The associate editor coordinating the review of this article and approving it for publication was Prof. Danda Rawat. *(Corresponding author: Kim Hammar.)*

Kim Hammar, Yuchao Li, and Dimitri P. Bertsekas are with the School of Computing and Augmented Intelligence, Arizona State University, Tempe, AZ 85281 USA (e-mail: khammar1@asu.edu; yuchaoli@asu.edu; dbertsek@asu.edu).

Tansu Alpcan is with the Department of Electrical and Electronic Engineering, The University of Melbourne, Melbourne, VIC 3010, Australia (e-mail: tansu.alpcan@unimelb.edu.au).

Emil C. Lupu is with the Department of Computing, Imperial College London, SW7 2BX London, U.K. (e-mail: e.c.lupu@imperial.ac.uk).

Digital Object Identifier 10.1109/TIFS.2026.3709131

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

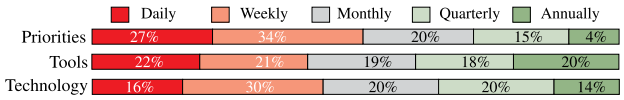


Fig. 2. Frequency of change in networked systems [46].

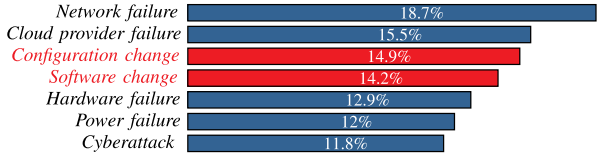


Fig. 3. Most common causes of outages in networked systems [47].

limited performance guarantees and requires extensive offline retraining to adapt. Moreover, most methods have only been tested in simulation, leaving their practical utility unvalidated.

In this paper, we address these limitations by presenting and validating a method for computing security policies that is scalable, offers performance guarantees, and adapts quickly to changes. Our method includes three main components, as shown in Fig. 1. First, we estimate a probabilistic belief about the system state through *particle filtering*. This belief quantifies the likelihood of potential system compromises and enables the security policy to account for uncertainty. Second, we aggregate the space of such beliefs into a finite set of representative ones, enabling efficient (offline) computation of a *base policy*, as well as approximation of the optimal cost function through dynamic programming (DP) [10]. We show that the error of this approximation is bounded. Third, we use (online) *rollout* [11] and lookahead optimization to adapt the base policy to changes in a given system model, such as changing operational conditions or security objectives; see Figs. 2 and 3. We show experimentally that this adaptation can be completed in seconds using commodity computing hardware and show theoretically that it is guaranteed to improve the base policy under general conditions.

Although our method is designed for security policies, it can be used more generally for adaptive control of partially observable dynamic systems. Compared to other approximation schemes for such systems [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], our method introduces a novel *feature-based* aggregation technique, which improves scalability and flexibility. Instead of aggregating beliefs over the state space directly, we first aggregate states into a small set of feature states and then aggregate beliefs over this set.

In this paper, we focus on control-theoretic system models where the attacker's strategy is fixed. If the attacker's strategy changes, our method adapts the policy to the change through rollout with an updated model. Extending our method to game-theoretic models where the attacker's strategy is coupled with the security policy is beyond the scope of this paper. Related aggregation methods for such models are discussed in [24].

We summarize our contributions as follows:

- We develop a scalable method for computing adaptive network security policies, which involves a novel combination of particle filtering, aggregation, and rollout.
- We establish a bound on the approximation error of the cost function obtained through our aggregation method.
- We show conditions under which our rollout method for policy adaptation improves the security policy.
- We evaluate our method through simulations and testbed experiments. The results show state-of-the-art performance on several benchmarks, including CAGE-2.

TABLE I

COMPARISON WITH RELATED WORK. OUR METHOD IS THE FIRST TO BE VALIDATED ON A TESTBED AND ACHIEVE STATE-OF-THE-ART (SOTA) RESULTS ON CAGE-2 [28] WHILE PROVIDING THEORETICAL GUARANTEES AND ADAPTING QUICKLY TO CHANGES

Method	*CAGE-2 sota*	*Adaptive*	*Testbed*	*Formal guarantees*
Ours (Fig. 1)	✓	✓	✓	✓
Deep RL [29], [30]	✓	✗	✗	✗
Deep RL [31]–[36]	✗	✗	✗	✗
Evolution [37]	✗	✗	✗	✗
Deep MARL [38]–[40]	✗	✗	✗	✗
LLM [41], [42], [45]	✗	✗	✗	✗
Tree search [43]	✓	✓	✗	✓

II. RELATED WORK

The problem of computing security policies has engaged security experts, control engineers, and game theorists for over two decades [25]. Surveys [3], [26], and [27] give an extensive account of these efforts and an appraisal of the state of the art. A driving factor behind this research is the development of evaluation benchmarks, which allow researchers to compare different methods. Currently, the most popular benchmark is the cyber autonomy gym for experimentation 2 (CAGE-2) [28], which involves computing an intrusion response policy.

More than 35 methods have been evaluated against CAGE-2 [28]. Detailed descriptions of some methods can be found in [29], [30], [31], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], [42], [43], [44], [45]. Although good results have been obtained, key aspects remain unexplored. For example, current methods focus narrowly on offline (deep) RL, which adapts slowly to changes and lacks performance guarantees. One exception is the method based on tree search presented in [43, Alg. 1]. However, this method is customized for a specific system and not generalizable. None of the current methods considers aggregation and rollout, which we introduce in this paper. The benefit of our approach is that it provides performance guarantees and adapts policies quickly to changes after the system model is updated. Another difference is that the referenced methods are evaluated only in simulation, whereas we evaluate our method both in simulation and in a testbed; see Table I.

Beyond methods evaluated on CAGE-2, our method is related to other control-theoretic approaches in cybersecurity. For example, Aydeger et al. [48] and Zhu et. al. [49] use adaptive control techniques for moving target defense; Hammar et al. [21], [50] employ model-predictive control for incident response; Miehling et al. [51] and Zonouz et al. [5] use tree search for attack mitigation; and Yue et al. [52] use game theory for security decision-making in control systems.

Our method differs from these methods in three main ways. First, the referenced methods are each designed for a specific security application. By contrast, our method is applicable to

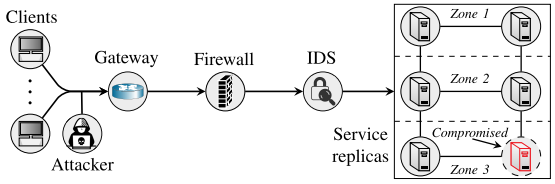


Fig. 4. Architecture of the networked system in the example use case.

any security problem that can be cast as a finite-state partially observable Markov decision problem. Second, we introduce a novel feature-based aggregation technique, which improves scalability. Third, we demonstrate this scalability by evaluating our method on CAGE-2, a system of considerably greater complexity than the systems considered in the referenced studies. Whether the methods proposed in those studies could scale to systems of comparable size remains an open question.

Lastly, we note that our method draws on ideas from model-predictive control (see e.g., Borrelli et al. [53]) and adaptive control (see e.g.,Åström and Wittenmark [54]), both of which are well-developed research areas with a long history.

III. Example Use Case: Intrusion Recovery

To illustrate the need for adaptive security policies, consider the networked system in Fig. 4. This system consists of service replicas that collectively provide services to a client population through a public gateway. Though intended for service delivery, this gateway is also accessible to a potential attacker who may compromise replicas. To maintain service to the clients even in the face of such attacks, the system can recover a replica suspected of being compromised by restarting it from a new virtual machine image. The decision whether to recover a replica or not is governed by a *security policy* based on alerts generated by an intrusion detection system (IDS).

A key challenge when designing this policy is that the distribution of alerts depends on many factors that change over time, such as the service load and the system configuration. As a result, the policy must be frequently adapted to remain effective. Without adaptation, the policy risks either overlooking attacks or overreacting to benign alerts, both of which degrade system performance and incur operational costs.

We formalize the problem of adapting such policies in the next section, after which we present our approach.

IV. Problem Formulation

We formulate the problem of obtaining an effective security policy for a networked system as a partially observable Markov decision problem (POMDP). Following this formalism, a security policy is a function that sequentially prescribes *controls* (i.e., security measures) based on a series of *observations* (e.g., system metrics). These controls stochastically influence the evolution of the system's *state*, which captures its security and service status. Due to limited monitoring capabilities or intentional concealment by a potential attacker, the state of the system cannot be observed directly. Therefore, controls are selected based on a state of *belief*, which represents the conditional probability distribution over possible states of the system given observations. The effectiveness of these controls is measured with respect to a specified objective, which is quantified through a *cost function* that should be minimized.

We denote the set of controls by U, the set of observations by Z, and the set of states by $X = \{1, \ldots, n\}$, all of which are finite. State transitions $i \to j$ under control u occur at discrete times k according to transition probabilities $p_{ij}(u)$. Each transition is associated with a cost $g(i, u, j) \in \Re$ and an observation z, which is generated with probability $p(z \mid j, u)$.

While the POMDP involves imperfect state information, it can be formulated as an equivalent problem with perfect state information [55]. In this formulation, the system is described by the belief state $b = \big(b(1), \ldots, b(n)\big)$, where $b(i)$ is the conditional probability that the state is i, given the history of controls and observations. This vector belongs to the belief space B and is updated through a belief estimator F as

$$b_k = F(b_{k-1}, u_{k-1}, z_k). \tag{1}$$

We adopt the belief-space formulation and consider security policies μ that map the belief space B to the control space U. The cost function of such a policy is defined as

$$J_\mu(b_0) = \lim_{N\to\infty} E\left\{\sum_{k=0}^{N-1} \alpha^k \hat{g}\big(b_k, \mu(b_k)\big)\right\}, \tag{2}$$

where $E\{\cdot\}$ denotes the expected value, $\alpha \in (0, 1)$ is a discount factor, and the stage cost $\hat{g}(b, u)$ is defined as

$$\hat{g}(b, u) = \sum_{i=1}^{n} b(i) \sum_{j=1}^{n} p_{ij}(u) g(i, u, j). \tag{3}$$

The optimal cost function $J^\star$, derived by optimizing over all possible policies μ, uniquely satisfies the Bellman equation

$$J^\star(b) = \min_{u\in U}\left[\hat{g}(b, u) + \alpha \sum_{z\in Z} \hat{p}(z \mid b, u) J^\star(F(b, u, z))\right], \tag{4}$$

where the probability $\hat{p}(z \mid b, u)$ is defined as

$$\hat{p}(z \mid b, u) = \sum_{i=1}^{n} b(i) \sum_{j=1}^{n} p_{ij}(u) p(z \mid j, u). \tag{5}$$

We say that a policy $\mu^\star$ is optimal if $J_{\mu^\star} = J^\star$. Although such a policy exists (see e.g., [56, Thm. 7.6.1] or [10, § 5.6]), there are no efficient algorithms to obtain it. Consequently, approximations are required in practice. A further complication is that the POMDP model may change over time due to changes in the networked system. When such changes occur, the policy must be adapted to remain effective.

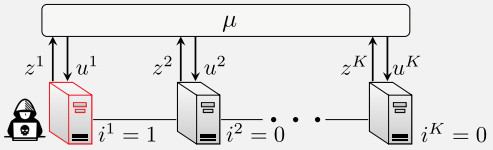
We use the following POMDP as a running example.

V. Our Method for Computing Security Policies

Building on the above problem formulation, we develop a method for approximating optimal security policies. It consists of three components: (*i*) belief estimation through particle filtering; (*ii*) offline policy computation through aggregation [10]; and (*iii*) online policy adaptation through rollout [11].

Example POMDP: Intrusion recovery

Consider the recovery use case described in §III, which involves a networked system with K service replicas. Each replica has two states: 1 (compromised) or 0 (safe), i.e., $i = (i^1, \ldots, i^K)$ where $i^l \in \{0, 1\}$. Compromises occur randomly over time and incur operational costs. Intrusion detection systems generate security alerts $z = (z^1, \ldots, z^K)$ that provide partial indications of the replicas' states. The security policy μ prescribes the control vector $u = (u^1, \ldots, u^K)$, where each u^l determines whether to recover component l ($u^l = 1$) or take no action ($u^l = 0$). The goal is to determine an optimal recovery policy $\mu^\star$ that balances security requirements against recovery costs. (Further details about this POMDP are provided in §VI.)



A. Belief Estimation Through Particle Filtering

In a security context, the belief state represents a probabilistic estimate of the system's security state. Consequently, accurate belief estimation is key to making informed security decisions amidst uncertainty about potential attacks.

The belief state can be computed via the following recursion

$$b_k(j) = \frac{p(z_k \mid j, u_{k-1}) \sum_{i=1}^{n} b_{k-1}(i) p_{ij}(u_{k-1})}{\sum_{i'=1}^{n} \sum_{j'=1}^{n} p(z_k \mid j', u_{k-1}) b_{k-1}(i') p_{i'j'}(u_{k-1})}. \quad (6)$$

However, the complexity of this calculation is quadratic in the number of states n, which typically grows exponentially with the number of system components; see, e.g., the example POMDP. For this reason, we estimate the belief state by

$$\hat{b}_k(j) = \frac{1}{M} \sum_{s=1}^{M} \delta_{j\hat{j}_k^s}, \quad \text{for all } j \in X, \quad (7)$$

where $\delta_{ij} = 1$ if $i = j$ and $\delta_{ij} = 0$ if $i \neq j$. The states (particles) $\hat{j}_k^1, \ldots, \hat{j}_k^M$ are sampled with probability proportional to the numerator in Eq. (6), and M is the number of particles. Such sampling ensures that the estimate $\hat{b}_k$ converges (almost surely) to b_k when $M \to \infty$; see e.g., [57]. Hence, Eq. (7) provides a consistent way to estimate beliefs while allowing computational cost to be adjusted by tuning M.

B. Offline Policy Computation Through Belief Aggregation

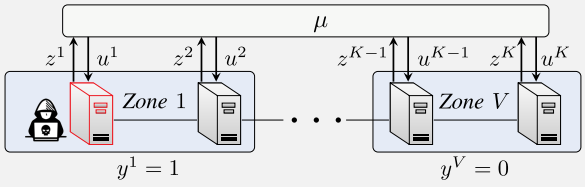
While the particle filter enables efficient estimation of beliefs [cf. Eq. (1)], the problem of computing an optimal policy remains intractable. We address this challenge by aggregating the belief space into a finite set of *representative beliefs*. Through this aggregation, we construct an aggregate Markov decision problem (MDP) with a finite state space, whose solution can be used to approximate that of the POMDP.

The aggregation consists of two stages. First, we aggregate the states $i \in X$ into *feature states* $y \in \mathcal{F}$, where the feature space $\mathcal{F}$ is smaller than X. Our method does not restrict how $\mathcal{F}$ is constructed; it accommodates both hand-crafted features based on engineering intuition and automatic feature extraction through data-driven approaches. We illustrate the construction of the feature space $\mathcal{F}$ in the gray box to the right.

After aggregating states into feature states, the second stage of our aggregation method involves grouping beliefs over the feature space $\mathcal{F}$. We denote these beliefs by q and the corresponding feature belief space by Q. We aggregate them via discretization into a finite set of *representative feature beliefs* $\tilde{Q} \subset Q$, whose elements are written as $\tilde{q}$. This two-stage aggregation is illustrated conceptually in Fig. 5.

Example feature space for aggregation

In the context of our running example, the feature space $\mathcal{F}$ can be obtained by grouping the K service replicas based on their network zone. Specifically, we can define a feature state as $y = (y^1, \ldots, y^V)$, where $V < K$ is the number of zones and $y^v = 1$ if any replica in zone v is compromised and $y^v = 0$ otherwise. This yields a feature space of size 2^V, which can be substantially smaller than the number of states, which is $n = 2^K$ in this example.



The first aggregation stage (the left arrow in Fig. 5) involves connecting states $i \in X$ with feature states $y \in \mathcal{F}$. We specify this connection as follows.

- With every feature state $y \in \mathcal{F}$, we associate a subset $I_y \subset X$. We require that the sets $(I_y)_{y \in \mathcal{F}}$ are disjoint.
- With every feature state $y \in \mathcal{F}$, we associate its *disaggregation probability distribution* $\{d_{yi} \mid i \in X\}$. We require that $d_{yi} = 0$ for all $i \notin I_y$.
- With every state $j \in X$, we associate its *aggregation probability distribution* $\{\phi_{jy} \mid y \in \mathcal{F}\}$. We require that $\phi_{jy} = 1$ for all $j \in I_y$ and $y \in \mathcal{F}$.

The second aggregation stage (the right arrow in Fig. 5) involves specifying a finite set of beliefs over the feature space $\mathcal{F}$. We construct such a set via uniform discretization as

$$\tilde{Q} = \left\{ \tilde{q} \;\middle|\; \tilde{q} \in Q, \tilde{q}(y) = \frac{\beta_y}{\rho}, \sum_{y \in \mathcal{F}} \beta_y = \rho, \beta_y \in \{0, \ldots, \rho\} \right\}, \quad (8)$$

where Q denotes the belief space over $\mathcal{F}$ and $\rho \in \{1, 2, \ldots\}$ can be interpreted as the *discretization resolution*. We refer to the elements of this subset as *representative feature beliefs*.

Approximation of a policy and cost function for the POMDP. We now use the set of representative feature beliefs $\tilde{Q}$ [cf. Eq. (8)], the disaggregation probabilities d_{yi}, and the aggregation probabilities ϕ_{iy} to construct a (computationally tractable) *aggregate MDP* whose solution can be used to approximate that of the original POMDP.

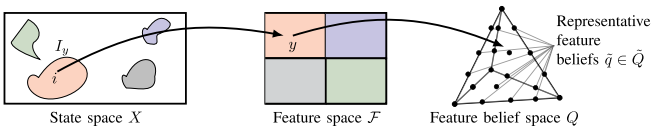


Fig. 5. Feature-based belief aggregation: we map the state space X into a feature space $\mathcal{F}$, over which beliefs are aggregated via discretization. In this illustration, a subset of states is mapped to a feature space with 4 elements, where I_y denotes the set of states that aggregate to feature state $y \in \mathcal{F}$. The resulting feature belief space Q is the 3-dimensional unit-simplex.

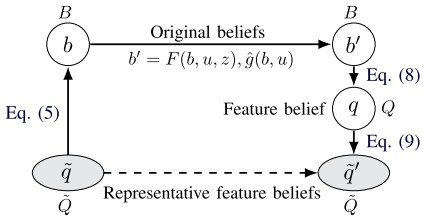


Fig. 6. One-step transition diagram of the aggregate MDP with finite state space $\tilde{Q}$ constructed through our feature-based belief aggregation method.

The aggregate MDP starts from a representative feature belief $\tilde{q} \in \tilde{Q}$ and evolves as follows. First, it transitions from $\tilde{q}$ to a belief $b \in B$ via the disaggregation probabilities as

$$b(i) = \sum_{y \in \mathcal{F}} \tilde{q}(y) d_{yi}, \quad \text{for all } i \in X. \tag{9}$$

Subsequently, a control $u \in U$ is applied, which generates an observation $z \in Z$ according to Eq. (5) and incurs a cost $\hat{g}(b, u)$ according to Eq. (3). The belief b is then updated as $b' = F(b, u, z)$ [cf. Eq. (1)], after which the MDP transitions to a feature belief $q \in Q$ via the aggregation probabilities as

$$q(y) = \sum_{i=1}^{n} b'(i) \phi_{iy}, \quad \text{for all } y \in \mathcal{F}. \tag{10}$$

Finally, the resulting feature belief q is mapped to a *representative* feature belief $\tilde{q}' \in \tilde{Q}$ via the nearest-neighbor mapping

$$\tilde{q}' \in \arg\min_{\tilde{q} \in \tilde{Q}} \|q - \tilde{q}\|, \tag{11}$$

where tie-breaking is consistent and $\|\cdot\|$ is the maximum norm. From the new representative feature belief $\tilde{q}' \in \tilde{Q}$, the MDP proceeds analogously by repeating the transitions in Eqs. (9)-(11). As a result, we obtain a well-defined MDP with state space $\tilde{Q}$ whose one-step transition diagram is shown in Fig. 6.

Due to the finite state space, the aggregate MDP in Fig. 6 can be efficiently solved using DP or RL. Let $r^\star$ and $\pi^\star$ denote the optimal cost function and policy in this MDP, respectively. Further, let $\Phi : B \mapsto \tilde{Q}$ denote the mapping defined by Eqs. (10)-(11), i.e.,

$$\Phi(b) \in \arg\min_{\tilde{q} \in \tilde{Q}} \max_{y \in \mathcal{F}} \left| \tilde{q}(y) - \sum_{i=1}^{n} b(i) \phi_{iy} \right|, \tag{12}$$

where ties in the arg min are broken using a fixed rule.

Using this mapping, we approximate the optimal cost function $J^\star$ and policy $\mu^\star$ of the original POMDP as

$$\tilde{J}(b) = r^\star(\Phi(b)) \text{ and } \mu(b) = \pi^\star(\Phi(b)), \text{ for all } b \in B. \tag{13}$$

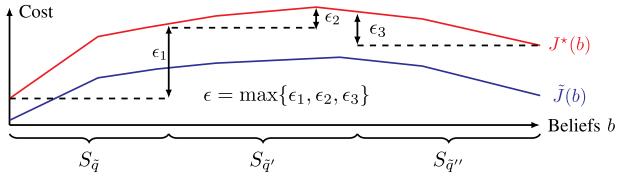


Fig. 7. Illustration of the scalar ϵ of Prop. 1. The illustration is based on an approximation with three representative feature beliefs: $\tilde{Q} = \{\tilde{q}, \tilde{q}', \tilde{q}''\}$. The corresponding belief space partitions are: $S_{\tilde{q}}$, $S_{\tilde{q}'}$, and $S_{\tilde{q}''}$; cf. Eq. (14).

We refer to the difference between the cost function approximation $\tilde{J}$ obtained through Eq. (13) and the optimal cost function $J^\star$ as the *approximation error*. To gain insight into this error, note that the aggregation mapping Φ [cf. Eq. (12)] partitions the belief space B into disjoint subsets $S_{\tilde{q}}$ as

$$B = \bigcup_{\tilde{q} \in \tilde{Q}} S_{\tilde{q}}, \quad \text{where } S_{\tilde{q}} = \{b \mid b \in B, \Phi(b) = \tilde{q}\}. \tag{14}$$

In view of Eq. (13), this partitioning means that the approximation error is determined by how much the optimal cost function $J^\star(b)$ varies for beliefs b within the same partition $S_{\tilde{q}}$. This insight is formalized by the following proposition.

Proposition 1 (Approximation error bound): The error of the cost function approximation in Eq. (13) is bounded as

$$|\tilde{J}(b) - J^\star(b)| \leq \frac{\epsilon}{1 - \alpha}, \quad \text{for all } b \in S_{\tilde{q}}, \tilde{q} \in \tilde{Q},$$

where ϵ is a finite constant defined by

$$\epsilon = \max_{\tilde{q} \in \tilde{Q}} \sup_{b, b' \in S_{\tilde{q}}} |J^\star(b) - J^\star(b')|.$$

A more general version of this proposition and additional auxiliary results are proved in [58]. Proposition 1 implies that the error of the approximation $\tilde{J}$ [cf. Eq. (13)] is small if the mapping Φ [cf. Eq. (12)] conforms to the optimal cost function $J^\star$ in the sense that Φ varies little in regions of the belief space where $J^\star$ also varies little. Hence, Prop. 1 provides a criterion to guide feature design: we seek a feature space $\mathcal{F}$, disaggregation probabilities d_{yi}, and aggregation probabilities ϕ_{iy} that induce belief space partitions $S_{\tilde{q}}$ [cf. Eq. (14)] over which $J^\star$ is approximately constant; see Fig. 7. A special case of interest, which we refer to as *identity aggregation*, is when each state is mapped to a unique feature state and vice versa. In this case, the cost function approximation in Eq. (13) converges to the optimal cost function $J^\star$ when the discretization resolution ρ in Eq. (8) is increased, as stated in the following proposition.

Proposition 2 (Asymptotic optimality): Suppose that $X = \mathcal{F}$ and $I_i = \{i\}$ for all states $i \in X$. We have

$$\lim_{\rho \to \infty} |\tilde{J}(b) - J^\star(b)| = 0, \quad \text{for all } b \in B.$$

This proposition implies that the approximation error vanishes under identity aggregation as the discretization resolution ρ increases. We present the proof of Prop. 2 in Appendix A.

From a network security viewpoint, the benefit of the preceding propositions is that they allow a network operator to weigh the trade-off between computational expenditure and

policy performance. For instance, the operator can allocate more resources and increase the discretization resolution in critical segments of the network to achieve finer security control where potential breaches carry severe consequences.

C. Online Policy Adaptation Through Rollout

The two-stage aggregation described above provides a scalable approach to *offline* policy computation. However, it does not provide a means for *online* policy adaptation. In a network security context, such adaptation is necessary as changes in networked systems occur regularly due to, e.g., evolving operational requirements and goals [46]; cf. Fig. 2.

For this reason, we complement the *base policy* μ computed offline via Eq. (13) with online lookahead optimization and *rollout* [11]. Specifically, at each step of online execution, we simulate the system's evolution several steps into the future, which allows us to evaluate different controls and adapt the base policy based on their outcomes. Effectively, these simulations can be understood by a security operator as a form of 'what if' analysis, where the system anticipates possible threats and assesses the impact of various security measures.

Mathematically, at each time step k during online execution, we transform the (pre-computed) base policy μ [cf. Eq. (13)] to a *rollout policy* $\tilde{\mu}$ via lookahead optimization as

$$\tilde{\mu}(b_k) \in \arg\min_{u_k \in U} \left[\hat{g}(b_k, u_k) + \min_{\mu_{k+1}, \ldots, \mu_{k+\ell-1}} \mathop{E}_{b_{k+1}, \ldots, b_{k+\ell}} \left\{ \sum_{j=k+1}^{k+\ell-1} \alpha^{j-k} \hat{g}(b_j, \mu_j(b_j)) + \alpha^{\ell} \tilde{J}_{\mu}(b_{k+\ell}) \right\} \right], \quad (15)$$

where $\ell \geq 1$ is the lookahead horizon, and the cost $\tilde{J}_{\mu}(b_{k+\ell})$ is estimated based on L simulations as

$$\tilde{J}_{\mu}(b_{k+\ell}) = \frac{1}{L} \sum_{s=1}^{L} \sum_{l=k+\ell}^{k+\ell+m-1} \alpha^{l-k-\ell} \hat{g}\left(b_l^s, \mu(b_l^s)\right) + \alpha^m \tilde{J}(b_{k+\ell+m}^s), \quad (16)$$

where $\tilde{J}$ is the cost function approximation in Eq. (13), m is the rollout horizon, the cost function $\hat{g}$ is defined in Eq. (3), and $(b_{k+\ell}^s, \ldots, b_{k+\ell+m}^s)$ is the belief trajectory of the sth simulation. Subsequently, the first control obtained through Eq. (15) is applied to the system, which yields an observation z that is used to update the belief through Eq. (1). The same computation is then repeated from the updated belief.

In a security context, a key property of the lookahead optimization in Eq. (15) is that the computational cost can be scaled by tuning the number of lookahead steps (ℓ) and the rollout horizon (m). This scalability enables our method to accommodate resource constraints that are common in operational systems. Another fundamental property of Eq. (15) is the *policy improvement* property, which is formalized below.

Proposition 3 (Policy improvement of the adaptation): If the policy evaluation in Eq. (16) is exact, i.e., if $\tilde{J}_{\mu} = J_{\mu}$, then the rollout policy $\tilde{\mu}$ improves the base policy μ, i.e., $J_{\tilde{\mu}} \leq J_{\mu}$. Further, the suboptimality of $\tilde{\mu}$ is bounded as

$$\|J_{\tilde{\mu}} - J^{\star}\| \leq \frac{2\alpha^{\ell}}{1-\alpha} \|\tilde{J}_{\mu} - J^{\star}\|.$$

The implication of this proposition is that our method can adapt *online* to changes in a given system model without repeating the offline computation. The proof follows directly from standard results by the last author; see [11, Prop. 2.3.1] and [59, Prop. 5.1.1] for details. We omit it here for brevity.

To apply the rollout method in Eq. (15), we require a model of the system, either in the form of an analytical model or a simulator. Identifying such a model is part of the broader *system identification* methodology; see e.g., Ljung [60]. When significant changes occur in the system (e.g., discovery of a new vulnerability), the identification must be repeated to update the model. The detailed updating procedure depends on the nature of the changes. In many cases, changes are structural and straightforward to incorporate. For instance, the deployment of a new service or the addition of a network link can be reflected directly in the model's structure. In other cases, changes may be more subtle, such as shifts in observation patterns, which may require statistical estimation to recalibrate the model's observation probabilities. Standard techniques for this purpose include maximum likelihood estimation and Bayesian learning; see e.g., Hammar [26, § 3.8]. We demonstrate an approach to such estimation in § VI-B.

While we require that the model be updated to reflect major changes to the system, the model does not have to be perfect for the rollout policy to be effective. Since the rollout policy is computed using DP principles, its robustness to model misspecification is related to that of other DP approaches. This robustness has been extensively analyzed in the robust control literature; see e.g., Iyengar [61]. Such an analysis is peripheral to the focus of the present paper and is therefore omitted here.

Lastly, we note that the requirement of an updated model is shared by other RL approaches. The key advantage of our method is that the offline policy optimization does not need to be repeated to update μ and $\tilde{J}$ in Eq. (16) for the new model. By contrast, methods based on deep RL require extensive offline retraining to adjust the policy after model updates [3].

D. Summary of Our Method for Computing Security Policies

In summary, our method for computing security policies is illustrated in Fig. 1 and involves three components:

1) *Offline policy computation via Eq. (13).*
   At the core of our method is the computation of a *base security policy* through DP in an aggregated belief space. This computation offers theoretical guarantees and scales to large systems.
2) *Online belief estimation via Eq. (7).*
   Our method uses network logs and metrics to estimate a probabilistic belief about the system state through *particle filtering*. This belief quantifies the likelihood of potential system compromises and serves as the basis for selecting appropriate security controls.
3) *Online policy adaptation via Eq. (15).*
   During online operation, our method adapts the base policy to system changes through lookahead optimization and rollout based on a given model. This procedure ensures that the adapted policy improves the base policy under general conditions.

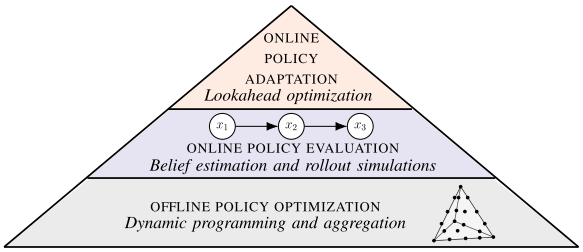


Fig. 8. The three computational layers of our method.

From an architectural point of view, our method extends current methods for computing security policies, which predominantly use offline (deep) RL [cf. § II], with online rollout and policy adaptation; see Fig. 8.

VI. Experimental Evaluation

In this section, we present an experimental evaluation of our method. We start by applying it to an instantiation of the example POMDP with a small state space, which allows us to illustrate Props. 1-3. We then assess the practical applicability of our method by applying it to an instantiation of the example POMDP with a practical system configuration in our testbed. Lastly, we evaluate the performance of our method on the CAGE-2 benchmark [28], which enables a comparison with state-of-the-art methods for computing network security policies. The experimental setup is described in Appendix B. Source code of our implementation is available at [62].

A. Numerical Illustrations Based on the Running Example

The example POMDP models a networked system with K service replicas; see § III. A state $i = (i^1, \ldots, i^K)$ of this POMDP represents the replicas' compromise statuses, where $i^l = 1$ if replica l is compromised and $i^l = 0$ otherwise. Similarly, a control $u = (u^1, \ldots, u^K)$ represents recovery actions, where $u^l = 1$ means to recover replica l and $u^l = 0$ means no recovery. We define the cost function as

$$g(i,u,j) = \sum_{l=1}^{K} \overbrace{w^l i^l (1-u^l)}^{\text{intrusion cost}} + \overbrace{u^l (1-i^l)}^{\text{recovery cost}}, \tag{17}$$

where w^l is a weighting factor for the intrusion cost of replica l. Hence, costs are incurred for unmitigated intrusions ($i^l = 1$) and unnecessary recovery actions ($u^l = 1$ and $i^l = 0$).

The observations of the POMDP correspond to the number of security alerts generated by an IDS. In the next section, we present an evaluation where these alerts are measured from an operational system. However, for the numerical illustrations presented in this section, we define the alert distribution as

$$p(z \mid i,u) = \prod_{l=1}^{K} p(z^l \mid i^l), \quad \text{for all } z \in Z, i \in X, u \in U,$$

where each $p(z^l \mid i^l)$ follows the Beta-binomial distribution shown in Fig. 9. This distribution reflects that alerts may occur during normal operation but are more likely during attacks; see Appendix B for details about this distribution.

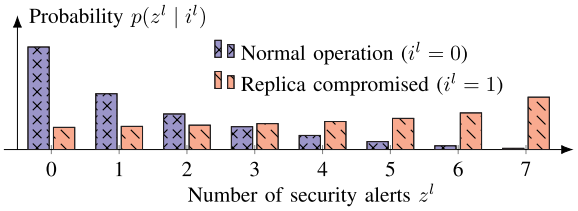


Fig. 9. Observation distribution for each replica l in the example POMDP.

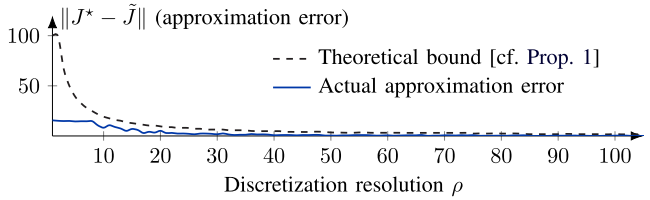


Fig. 10. Comparison between the theoretical error bound in Prop. 1 and the actual error of the approximation $\tilde{J}$ [cf. Eq. (13)] when applied to the example POMDP with $K = 1$ and varying discretization resolutions ρ; cf. Eq. (8).

The transition probabilities $p_{ij}(u)$ are defined as follows. If replica l is compromised ($i^l = 1$), then it remains so until recovery is applied ($u^l = 1$), at which point the state i^l is set to 0. Otherwise, the probability that it becomes compromised is $\min\{0.2(1 + \mathcal{N}_l(i)), 1\}$, where $\mathcal{N}_l(i)$ is the number of compromised neighbors of replica l in the network.

Instantiation of our method. We instantiate our method for the example POMDP with the zone-based feature space defined in § V-B and different discretization resolutions ρ; cf. Eq. (8). Specifically, we group the replicas into $V < K$ zones and define a feature state as $y = (y^1, \ldots, y^V)$, where $y^v = 1$ if any replica in zone v is compromised and $y^v = 0$ otherwise. This yields a feature space $\mathcal{F}$ of size 2^V, which can be substantially smaller than the number of states, which is $n = 2^K$. We assume that the network follows the line topology shown in § V-B and that neighboring replicas are grouped into zones of equal size. For example, if $K = 4$ and $V = 2$, then replicas 1 and 2 are in zone 1, whereas replicas 3 and 4 are in zone 2.

We define the aggregation probabilities as

$$\phi_{iy} = \begin{cases} 1 & \text{if } y^v = \max\{i^l \mid \text{replica } l \text{ is in zone } v\} \text{ for all } v, \\ 0 & \text{otherwise,} \end{cases}$$

for all states $i \in X$ and feature states $y \in \mathcal{F}$. Similarly, we define the disaggregation probabilities as

$$d_{yi} = \begin{cases} 1, & \text{if } i^l = y^v \text{for all replicas } l \text{ in each zone } v, \\ 0, & \text{otherwise,} \end{cases}$$

for all states $i \in X$ and feature states $y \in \mathcal{F}$. Finally, we define the weight in Eq. (17) to be $w^l = 2l$ for each replica l.

Numerical illustrations. We start by analyzing how close the bound in Prop. 1 is to the actual approximation error, i.e., $\|\tilde{J} - J^\star\|$. Although this quantity is generally intractable to evaluate due to its dependence on the optimal cost function $J^\star$, the case of a single replica ($K = 1$) yields a sufficiently small POMDP to permit exact computation of $J^\star$ via the incremental pruning algorithm [63]. As shown in Fig. 10, the bound is not

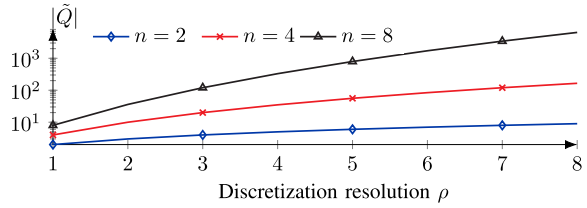


Fig. 11. Number of representative feature beliefs [cf. Eq. (8)] in function of the discretization resolution; curves relate to state spaces of different sizes.

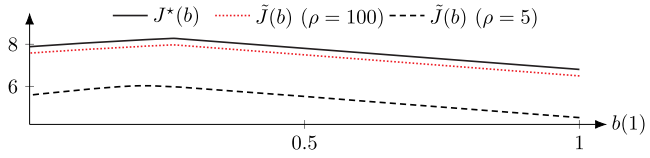


Fig. 12. Comparison between the optimal cost function $J^\star$ for the example POMDP and the approximation $\tilde{J}$ [cf. Eq. (13)]. The number of service replicas is $K = 1$. Hence, $b(1)$ is the belief of system compromise.

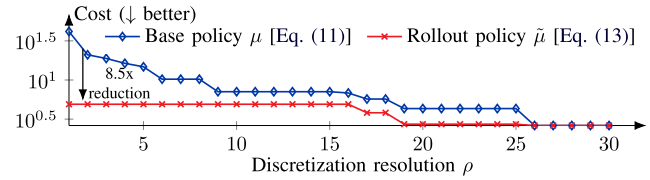


Fig. 13. Performance of rollout when applied to the example POMDP with $K = V = 4$. The rollout and lookahead horizons are $m = 10$ and $\ell = 1$. The base policy is computed with varying discretization resolution ρ; cf. Eq. (8).

tight but becomes increasingly accurate when the resolution ρ increases, as asserted in Prop. 2. However, increasing ρ also causes the number of representative feature beliefs to grow, which is illustrated in Fig. 11. Hence, ρ governs a trade-off between computational expedience and approximation error.

Next, Fig. 12 shows the structure of the optimal cost function $J^\star$ and the cost function approximation $\tilde{J}$; cf. Eq. (13). Interestingly, even when the difference between them is significant, they have a similar structure. We also note that $\tilde{J} \leq J^\star$. Although not guaranteed in our setup, $\tilde{J}$ can serve as a lower bound for $J^\star$ under certain conditions; see [58, Prop. 6].

We now turn our attention to Prop. 3. Figure 13 shows the performance of the base policy μ [cf. Eq. (13)] and the (adapted) rollout policy $\tilde{\mu}$ [cf. Eq. (15)] for varying discretization resolutions ρ. We observe that $\tilde{\mu}$ incurs a lower cost than μ, with the difference being dramatic in some cases. A theoretical explanation for the large cost reduction is provided by Bertsekas in [64], where it is shown that the rollout computation performs one step of Newton's method for solving Bellman's equation. As a consequence, if the base policy is sufficiently close to optimal, the rollout policy can exhibit a superlinear rate of convergence to the optimal, just like Newton's method in classical optimization.

We now analyze the quality of the zone-based feature space defined above. We assess its quality in two ways: computational scalability and approximation error. Figure 14 shows the offline computation time for the cost function approximation $\tilde{J}$ [cf. Eq. (13)] as a function of the number of replicas K

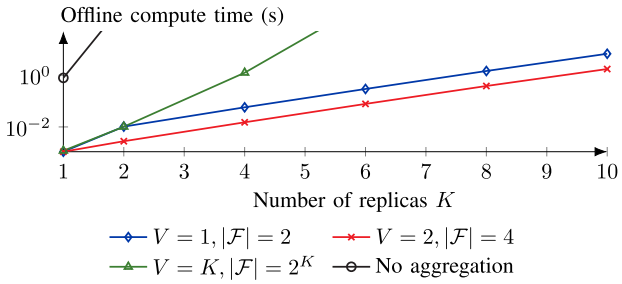


Fig. 14. Offline compute time to obtain the cost function approximation $\tilde{J}$ through Eq. (13) for the example POMDP with varying number of zones V and sizes of the feature space (i.e., varying $|\mathcal{F}|$). The discretization resolution is defined as $\rho = 2$; cf. Eq. (8). (The computation time for $V = K$ and no aggregation becomes impractical for large values of K.)

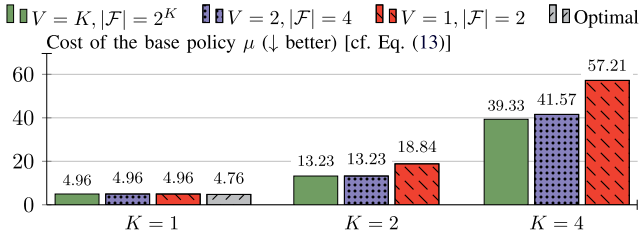


Fig. 15. Performance of the base policy μ [cf. Eq. (13)] when applied to the example POMDP with varying number of zones V and sizes of the feature space (i.e., varying $|\mathcal{F}|$). The discretization resolution is defined as $\rho = 2$; cf. Eq. (8). (Computing the optimal cost is impractical for $K > 1$.)

and zones V. When neither state nor belief aggregation is used, the computation becomes impractical for $K > 2$, which is expected due to the high complexity of solving POMDPs exactly. Similarly, when using only belief aggregation (i.e., $V = K$), the computation time grows exponentially with K and becomes impractical for $K > 5$. By contrast, when using both state and belief aggregation (i.e., $V < K$), the computation time remains manageable even as K increases. This demonstrates the scalability of feature-based aggregation.

However, the cost of this scalability is a higher approximation error, as shown in Fig. 15. We observe that the feature space obtained with $V = 2$ zones yields the best trade-off between scalability and approximation error. In particular, it leads to less than 5% approximation error compared to the optimal solution with $K = 1$ replicas and less than 6% error compared to the solution obtained with $K = V = 4$ replicas.

Automatic feature extraction. The zone-based feature space described above is motivated by engineering intuition. However, such intuition may not always suffice to construct effective features. In these situations, effective features can be learned directly from data using deep learning techniques. For example, given a simulator of the POMDP and a policy π, one can generate a dataset of state–cost pairs and train a neural network to approximate a function $H_\pi : X \mapsto \Re$, where the value $H_\pi(i)$ represents the expected cost incurred under policy π when the belief state is initialized as $b(i) = 1$. The learned function can then be used to inform the design of feature states, as originally proposed by the last author [15].

We apply this approach to the example POMDP as follows. We first generate state–cost pairs by simulating the POMDP under a threshold policy that recovers a replica whenever the belief of compromise exceeds 0.7. We then train a neural

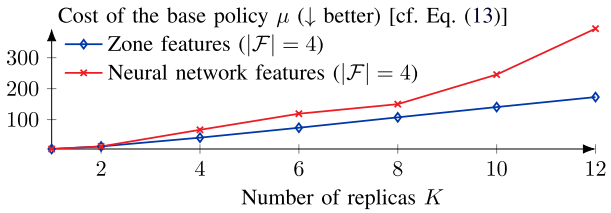


Fig. 16. Performance of the base policy μ [cf. Eq. (13)] when applied to the example POMDP with zone-based features and features learned through a neural network. The discretization resolution is defined as $\rho = 2$; cf. Eq. (8).

TABLE II

REPLICAS OF THE NETWORKED SYSTEM DESCRIBED IN § III. VULNERABILITIES ARE IDENTIFIED USING CVE [66] AND CWE [67] IDENTIFIERS

Replica	*Operating system*	*Background services*	*Vulnerabilities*
1	Deb 9.2	Apache2	CWE-89
2	Deb Jessie	FTP	CVE-2015-3306
3	Ubuntu 20	SSH, Spark	CWE-1391
4	Deb Jessie	Phpmailer	CVE-2016-10033
5	Deb Wheezy	NGINX	CVE-2014-6271
6	Deb Jessie	SSH, GRPC	CWE-1391, CVE-2010-0426
7	Deb Jessie	SSH, Spring boot	CVE-2015-5602, CWE-1391
8	Deb Jessie	Postgresql, Samba	CVE-2017-7494

network to approximate this policy's cost function. For the experiments reported in this paper, we use a neural network with three hidden layers of 64 neurons each and ReLU activations. From the trained network, we extract the output of the final hidden layer to obtain a feature vector for each state $i \in X$. To construct the feature space, we cluster these vectors using K-means with $|\mathcal{F}| = 4$ clusters [65], where each cluster defines a feature state y. Finally, we define the aggregation and disaggregation probabilities by assigning each state to the feature state that is nearest in Euclidean distance.

Figure 16 compares the cost of the base policy μ [cf. Eq. (13)] when using feature states derived via deep learning versus zone-based feature states. We observe that for small numbers of replicas (K), both approaches yield comparable performance. However, as K increases, the zone-based feature states lead to significantly lower cost.

B. Testbed Evaluation

We now complement the analytical and numerical evaluations presented in the previous sections by evaluating our method on an operational system. To facilitate this evaluation, we deploy the networked system described in § III on our testbed and subject it to a variety of cyberattacks. These attacks produce system measurements and logs, which we use to identify the parameters of the example POMDP. We then apply our method to the identified POMDP to compute a security policy. Finally, we deploy and execute the computed policy in our testbed and evaluate its performance against real cyberattacks, as well as its adaptability to system changes.

Testbed setup. We deploy the networked system described in § III on three physical servers in our testbed: two Supermicro 7049 and one Dell R740 2U. We run $K = 8$ virtual service replicas on these servers. Server 1 hosts replicas 1–4; server 2 hosts replicas 5 – 8; and server 3 emulates the cloud gateway. The configurations of the service replicas are listed in Table II and details of our testbed are available in Appendix C.

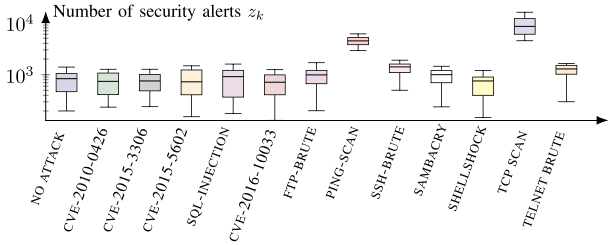


Fig. 17. Box plots of the empirical distributions (based on 24,386 measurements) of security alerts in our testbed under different attacks (indicated on the x-axis); see Appendix C for details about the attacks and the data collection. Each box represents the interquartile range (IQR) of the distribution, with the median shown as a horizontal line; vertical lines extend to points within 1.5 times the IQR. The empirical measurements are available at [62].

System identification. We identify the observation distribution of the example POMDP from measurement data obtained by running a sequence of emulated attacks and controls on our testbed. We define the length of a time step in our testbed to be 30 seconds. During each step, we execute attacks against a subset of replicas; see Appendix B for the list of attacks. We then measure the observation z_k from our testbed by reading log files. We repeat this procedure for 24,386 time steps and use the empirical distribution of security alerts over those time steps to define the observation distribution $p(z \mid i, u)$ in the POMDP. Figure 17 shows the estimated distribution.

Evaluation scenarios. We consider two evaluation scenarios, both of which represent executions of the example POMDP.

1) STATIONARY SYSTEM: In this scenario, the system operates under the same conditions throughout.
2) NON-STATIONARY SYSTEM: This scenario is divided into two time intervals. From time step $k = 0$ to $k = 200$, the system operates under the same conditions as in the first scenario. After time step $k = 200$, background processes are started on each service replica, which alters the alert distribution [cf. Fig. 17] and requires policy adaptation.

Methods for comparison. We compare our method with two baseline policies: (*i*) a periodic policy that recovers replicas every fifth time step; and (*ii*) a policy computed through PPO [68, Alg. 1], which is a popular method in related work.

Evaluation metrics. We compare the computed policies using three metrics: the cost in Eq. (17), the frequency with which the policies initiate recovery, and the average time-to-recovery, i.e., the average time from compromise to recovery initiation.

We compare the methods in terms of compute time and adaptation time, which refers to the time required to obtain a fully adapted policy. To evaluate the degree of adaptation of a policy μ, we use the adaptation-completion metric

$$A(\mu) = \frac{J_0(b_0) - J_\mu(b_0)}{J_0(b_0) - J_1(b_0)}, \tag{18}$$

where J_0 is the cost function at the start of adaptation and J_1 is the cost function of a fully adapted policy. Since the optimal cost is unknown, we define J_1 to be the cost of the best known policy. Here b_0 is the known initial belief, i.e.,

TABLE III

TESTBED RESULTS. NUMBERS IN THE LAST COLUMN INDICATE THE MEAN AND (STANDARD DEVIATION) FROM 5 EVALUATIONS. THE BEST RESULTS ARE IN BOLD

Method	*Offline compute*	*Online compute*	*Cost (↓ better)*
Scenario 1 results **(STATIONARY SYSTEM).**			
Ours, $m = 0, \ell = 1$	17 min	0.93 sec	34.61 (0.32)
Ours, $m = 0, \ell = 2$	17 min	12.87 sec	27.76 (0.38)
Ours, $m = 10, \ell = 1$	17 min	5.45 sec	34.35 (0.32)
Ours, $m = 10, \ell = 2$	17 min	16.41 sec	27.67 (0.38)
Ours, $m = 20, \ell = 1$	17 min	7.45 sec	23.42 (0.51)
Ours, $m = 20, \ell = 2$	17 min	19.31 sec	**20.12** (0.45)
Base policy [Eq. (13)]	17 min	0.01 sec	106.00 (0.32)
PPO [68, Alg. 1]	80 min	0.01 sec	**19.71** (9.32)
Periodic	0 min	0.01 sec	168.09 (0.22)
Scenario 2 results **(NON-STATIONARY SYSTEM).**			
Ours, $m = 0, \ell = 1$	17 min	0.93 sec	38.83 (1.12)
Ours, $m = 0, \ell = 2$	17 min	12.87 sec	29.76 (0.53)
Ours, $m = 10, \ell = 1$	17 min	5.45 sec	31.31 (0.51)
Ours, $m = 10, \ell = 2$	17 min	16.41 sec	26.67 (0.53)
Ours, $m = 20, \ell = 1$	17 min	7.45 sec	23.61 (0.69)
Ours, $m = 20, \ell = 2$	17 min	19.31 sec	**20.92** (0.48)
Base policy [Eq. (13)]	17 min	0.01 sec	114.48 (0.32)
PPO [68, Alg. 1]	80 min	0.01 sec	49.71 (13.67)
Periodic	0 min	0.01 sec	168.09 (0.22)

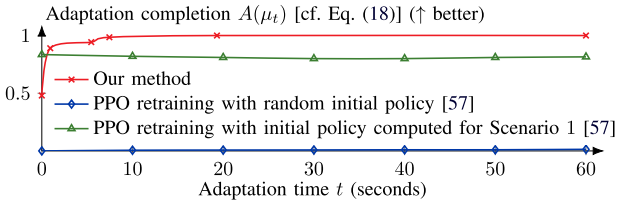


Fig. 18. Policy adaptation time after a system change in our testbed (Scenario 2). The metric $A(\mu_t)$ is calculated according to Eq. (18) with $J_1(b_0) = 20.92$, where μ_t is the policy after t seconds of adaptation.

$J_\mu(b_0)$ is the expected cost of policy μ at the start of the evaluation.

Instantiation of our method. We use identity aggregation (i.e., $X = \mathcal{F}$) with discretization resolution $\rho = 2$, which leads to 32,896 representative feature beliefs; cf. Eq. (8).

Evaluation results. The compute times and the cost values of each method are listed in Table III. We observe that our method and PPO [68] achieve the lowest cost for Scenario 1, significantly outperforming the baselines. However, the results of PPO exhibit a higher variability than our method, which we attribute to PPO's tendency to converge to different local optima. Moreover, PPO requires four times more offline compute time than our method. We also note that the performance of our method improves when increasing the rollout and lookahead horizons (m and ℓ), at the expense of more online compute.

In the results from Scenario 2, we observe that our method outperforms all other methods. We explain this improvement by our method's ability to adapt policies to changes. The adaptation time of our method and the baselines are shown in Fig. 18. We find that effective policy adaptation takes 19 seconds with our method and computing hardware, compared to more than 30 minutes with PPO using the same hardware.

Lastly, Fig. 19 shows the average time-to-recovery and the recovery frequency of the policies. We observe that the policies computed by our method and PPO achieve the lowest time-to-recovery, indicating that they initiate recovery more promptly after compromise. At the same time, they maintain a low

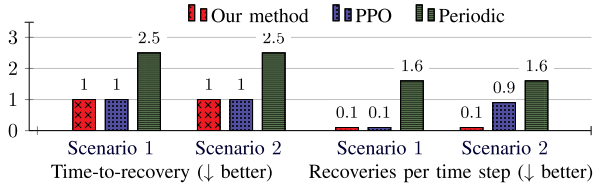


Fig. 19. Results from the testbed evaluation. The time-to-recovery and recovery frequency are averaged across 1000 time steps.

Takeaway from the testbed evaluation.

When applied to the use case in §III, our method yields security policies that recover more effectively from attacks than periodic recovery and adapt faster to system changes than those learned with PPO.

recovery frequency when compared to the periodic policy, which also has a high time-to-recovery. However, the recovery frequency of PPO increases significantly in Scenario 2.

C. Cage-2 Evaluation

To compare our method with the state-of-the-art methods for computing security policies, we apply it to the CAGE-2 benchmark [28]. CAGE-2 involves a (simulated) networked system segmented into *zones* with *nodes* (servers and workstations) offering services to clients through a gateway, which is also accessible to an attacker; see Fig. 20. The system emits network statistics, which the security policy μ uses to prescribe security controls. These controls are applied to specific nodes of the system and can be grouped into four categories: intrusion analysis, decoy deployment, malware removal, or secure reset (which disrupts service). Each service disruption and node compromise incurs a predefined cost; the problem is to find a security policy that minimizes this cost. When formulated as a POMDP, CAGE-2 has 145 controls, over 10^{47} states, and over 10^{25} observations [43].

Methods for comparison. Over 35 methods have been evaluated against the CAGE-2 benchmark. We compare our method against the current state-of-the-art methods, namely: Cardiff [29] and C-POMCP [43, Alg. 1]. We also compare it against four baseline methods: PPO [68, Alg. 1], PPG [69, Alg. 1], DQN [70, Alg. 1], and POMCP [71, Alg. 1].

Evaluation scenarios.

1) STATIONARY SYSTEM: This is the standard CAGE-2 scenario where the system dynamics are stationary.
2) NON-STATIONARY SYSTEM: This scenario is divided into two time intervals. In the first interval ($[0, 20]$), the system behaves as in the stationary case. In the second interval, which starts at $k = 20$, the decoys become ineffective (e.g., known to the attacker). As a consequence, the security policy must be adapted to remain effective.

Evaluation metrics. We compare the computed policies in terms of cost (calculated by the CAGE-2 simulator) and further

TABLE IV

EVALUATION RESULTS ON CAGE-2 (SCENARIO 1). ROWS RELATE TO DIFFERENT METHODS; COLUMNS INDICATE PERFORMANCE METRICS AND CONFIGURATIONS; GREEN ROWS RELATE TO OUR METHOD (SEE FIG. 1); BLUE ROWS RELATE TO THE PREVIOUS STATE-OF-THE-ART METHODS; RESULTS THAT ARE WITHIN THE MARGIN OF STATISTICAL EQUIVALENCE TO THE STATE-OF-THE-ART ARE HIGHLIGHTED IN BOLD (↓ BETTER); NUMBERS IN THE LAST COLUMN INDICATE THE MEAN AND THE (STANDARD DEVIATION) FROM 1000 EVALUATIONS. THE COST IS CALCULATED USING CAGE-2'S INTERNAL COST FUNCTION (COMMIT 9421C8E) WITH THE B-LINE ATTACKER AND 100 TIME STEPS. (SINCE CURRENT DEEP RL METHODS DO NOT USE BELIEF STATES, WE REPORT THEIR PERFORMANCE BOTH WITH AND WITHOUT THE PARTICLE FILTER IN EQ. (7) TO ENABLE A FAIR COMPARISON)

Method	*Rollout* m	*Offline/Online compute (min/s)*	*State estimation*	*Lookahead* ℓ	*Base policy* μ	*Cost (↓ better)*
μ [Eq. (13)]	-	8.5/0.01	Particle filter [Eq. (7)]	-	-	15.19 (0.82)
PPO [68, Alg. 1]	-	1000/0.01	Latest observation	-	-	280 (114)
PPO [68, Alg. 1]	-	1000/0.01	Particle filter [Eq. (7)]	-	-	119 (58)
PPG [69, Alg. 1]	-	1000/0.01	Latest observation	-	-	338 (147)
PPG [69, Alg. 1]	-	1000/0.01	Particle filter [Eq. (7)]	-	-	299 (108)
DQN [70, Alg. 1]	-	1000/0.01	Latest observation	-	-	479 (267)
DQN [70, Alg. 1]	-	1000/0.01	Particle filter [Eq. (7)]	-	-	462 (244)
Cardiff [29]	-	300/0.01	Latest observation	-	-	**13.69** (0.53)
Cardiff [29]	-	300/0.01	Particle filter [Eq. (7)]	-	-	**13.31** (0.87)
POMCP [71, Alg. 1]	-	0/0.05	Particle filter [Eq. (7)]	-	Random	38.71 (2.0)
POMCP [71, Alg. 1]	-	0/0.1	Particle filter [Eq. (7)]	-	Random	38.02 (0.53)
POMCP [71, Alg. 1]	-	0/0.5	Particle filter [Eq. (7)]	-	Random	34.92 (0.96)
POMCP [71, Alg. 1]	-	0/1	Particle filter [Eq. (7)]	-	Random	34.50 (0.65)
POMCP [71, Alg. 1]	-	0/5	Particle filter [Eq. (7)]	-	Random	33.06 (0.21)
POMCP [71, Alg. 1]	-	0/15	Particle filter [Eq. (7)]	-	Random	30.88 (1.41)
POMCP [71, Alg. 1]	-	0/30	Particle filter [Eq. (7)]	-	Random	29.51 (2.00)
C-POMCP [43, Alg. 1]	-	0/0.05	Particle filter [Eq. (7)]	-	Random	25.05 (3.02)
C-POMCP [43, Alg. 1]	-	0/0.1	Particle filter [Eq. (7)]	-	Random	21.28 (0.72)
C-POMCP [43, Alg. 1]	-	0/0.5	Particle filter [Eq. (7)]	-	Random	18.08 (1.32)
C-POMCP [43, Alg. 1]	-	0/1	Particle filter [Eq. (7)]	-	Random	17.42 (1.08)
C-POMCP [43, Alg. 1]	-	0/5	Particle filter [Eq. (7)]	-	Random	**13.23** (0.43)
C-POMCP [43, Alg. 1]	-	0/15	Particle filter [Eq. (7)]	-	Random	**12.98** (1.55)
C-POMCP [43, Alg. 1]	-	0/30	Particle filter [Eq. (7)]	-	Random	**13.32** (0.18)
Our method [Eq. (15)]	0	8.5/0.01	Particle filter [Eq. (7)]	1	μ [Eq. (13)]	**13.32** (0.65)
Our method [Eq. (15)]	0	8.5/0.95	Particle filter [Eq. (7)]	2	μ [Eq. (13)]	**13.24** (0.57)
Our method [Eq. (15)]	10	8.5/2.39	Particle filter [Eq. (7)]	1	μ [Eq. (13)]	**13.28** (0.72)
Our method [Eq. (15)]	10	8.5/8.29	Particle filter [Eq. (7)]	2	μ [Eq. (13)]	**13.23** (0.62)
Our method [Eq. (15)]	20	8.5/6.41	Particle filter [Eq. (7)]	1	μ [Eq. (13)]	**13.25** (0.78)
Our method [Eq. (15)]	20	8.5/14.80	Particle filter [Eq. (7)]	2	μ [Eq. (13)]	**13.23** (0.57)

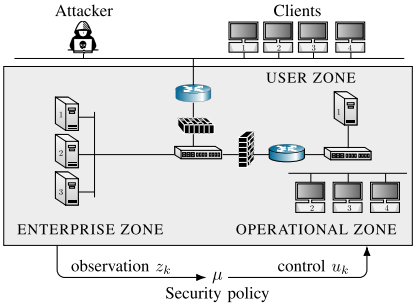


Fig. 20. The CAGE-2 benchmark problem [28]: compute a security policy μ to protect a system against an attacker while maintaining services for clients.

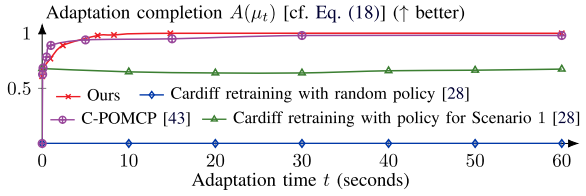


Fig. 21. Policy adaptation time after a system change in CAGE-2 (Scenario 2). The metric $A(\mu_t)$ is calculated according to Eq. (18) with $J_1(b_0) = 37.89$, where μ_t is the policy after t seconds of adaptation.

compare the methods in terms of compute and adaptation times, with adaptation time calculated using Eq. (18).

Instantiation of our method. The size of the state space in CAGE-2 exceeds 10^{47}, making it impractical to estimate beliefs over it. Therefore, we map each state into a *feature state* with three components: ATTACKER-STATE, ATTACKER-TARGET, and DECOY-STATE. The first two components represent the attacker's location in the network and its target node. The last component is the configuration of the decoys. These feature states lead to a feature belief space Q of dimension $427,500$, which we discretize with resolution $\rho = 1$; cf. Eq. (8). We define the aggregation probability ϕ_{jy} to be 1 if the feature state y is consistent with the state j and 0 otherwise. Finally, we define the disaggregation probabilities $\{d_{yi} \mid i \in X\}$ to be uniform over states i that are aggregated to the feature state y, i.e., states where $\phi_{iy} = 1$.

***Scenario 1 results* (STATIONARY SYSTEM).** The results are presented in Table IV. We observe that our method achieves performance on par with the state-of-the-art methods in terms of cost. We also find that our method requires less offline compute time than methods based on deep RL (e.g., PPO, PPG, and DQN); see the third column of Table IV. Further, it requires less online compute time than POMCP [71] and C-POMCP [43]. The performance of our method improves only slightly when increasing the lookahead horizon ℓ from

TABLE V

EVALUATION RESULTS ON CAGE-2 (SCENARIO 2). ROWS RELATE TO DIFFERENT METHODS; COLUMNS INDICATE PERFORMANCE METRICS AND CONFIGURATIONS; GREEN ROWS RELATE TO OUR METHOD (SEE FIG. 1); BLUE ROWS RELATE TO THE PREVIOUS STATE-OF-THE-ART METHODS; THE BEST RESULTS ARE HIGHLIGHTED IN BOLD (↓ BETTER); NUMBERS IN THE LAST COLUMN INDICATE THE MEAN AND THE (STANDARD DEVIATION) FROM 1000 EVALUATIONS. THE COST IS CALCULATED USING CAGE-2'S INTERNAL COST FUNCTION (COMMIT 9421C8E) WITH THE B-LINE ATTACKER AND 100 TIME STEPS

Method	*Rollout* m	*Offline/Online compute (min/s)*	*State estimation*	*Lookahead* ℓ	*Base policy* μ	*Cost (↓ better)*
μ [Eq. (13)]	-	8.5/0.01	Particle filter [Eq. (7)]	-	-	61.72 (3.96)
PPO [68, Alg. 1]	-	1000/0.01	Latest observation	-	-	341 (133)
PPO [68, Alg. 1]	-	1000/0.01	Particle filter [Eq. (7)]	-	-	326 (116)
PPG [69, Alg. 1]	-	1000/0.01	Latest observation	-	-	328 (178)
PPG [69, Alg. 1]	-	1000/0.01	Particle filter [Eq. (7)]	-	-	312 (163)
DQN [70, Alg. 1]	-	1000/0.01	Latest observation	-	-	516 (291)
DQN [70, Alg. 1]	-	1000/0.01	Particle filter [Eq. (7)]	-	-	492 (204)
Cardiff [29]	-	300/0.01	Latest observation	-	-	57.45 (2.44)
Cardiff [29]	-	300/0.01	Particle filter [Eq. (7)]	-	-	56.45 (2.81)
POMCP [71, Alg. 1]	-	0/0.05	Particle filter [Eq. (7)]	-	Random	66.80 (4.80)
POMCP [71, Alg. 1]	-	0/0.1	Particle filter [Eq. (7)]	-	Random	57.12 (4.62)
POMCP [71, Alg. 1]	-	0/0.5	Particle filter [Eq. (7)]	-	Random	55.43 (3.99)
POMCP [71, Alg. 1]	-	0/1	Particle filter [Eq. (7)]	-	Random	53.71 (3.84)
POMCP [71, Alg. 1]	-	0/5	Particle filter [Eq. (7)]	-	Random	52.23 (3.81)
POMCP [71, Alg. 1]	-	0/15	Particle filter [Eq. (7)]	-	Random	53.08 (3.78)
POMCP [71, Alg. 1]	-	0/30	Particle filter [Eq. (7)]	-	Random	53.18 (3.42)
C-POMCP [43, Alg. 1]	-	0/0.05	Particle filter [Eq. (7)]	-	Random	61.05 (5.84)
C-POMCP [43, Alg. 1]	-	0/0.1	Particle filter [Eq. (7)]	-	Random	57.46 (4.53)
C-POMCP [43, Alg. 1]	-	0/0.5	Particle filter [Eq. (7)]	-	Random	51.18 (4.37)
C-POMCP [43, Alg. 1]	-	0/1	Particle filter [Eq. (7)]	-	Random	44.52 (3.75)
C-POMCP [43, Alg. 1]	-	0/5	Particle filter [Eq. (7)]	-	Random	41.61 (3.34)
C-POMCP [43, Alg. 1]	-	0/15	Particle filter [Eq. (7)]	-	Random	40.84 (2.92)
C-POMCP [43, Alg. 1]	-	0/30	Particle filter [Eq. (7)]	-	Random	**38.71** (2.38)
Our method [Eq. (15)]	0	8.5/0.01	Particle filter [Eq. (7)]	1	μ [Eq. (13)]	58.23 (1.67)
Our method [Eq. (15)]	0	8.5/0.95	Particle filter [Eq. (7)]	2	μ [Eq. (13)]	51.87 (1.42)
Our method [Eq. (15)]	10	8.5/2.39	Particle filter [Eq. (7)]	1	μ [Eq. (13)]	44.38 (1.76)
Our method [Eq. (15)]	10	8.5/8.29	Particle filter [Eq. (7)]	2	μ [Eq. (13)]	**38.81** (1.68)
Our method [Eq. (15)]	20	8.5/6.41	Particle filter [Eq. (7)]	1	μ [Eq. (13)]	**39.05** (2.04)
Our method [Eq. (15)]	20	8.5/14.80	Particle filter [Eq. (7)]	2	μ [Eq. (13)]	**37.89** (1.54)

1 to 2 and when increasing the rollout horizon m in Eq. (15) from 0 to 20; see the second and fifth columns in Table IV. This suggests that the performance achieved with $\ell = 1$ and $m = 0$ may be near optimal. Finally, we note that the results of PPO, DQN, and PPG exhibit high variability. This variability may be due to their tendency to converge to different local optima depending on the random seed.

***Scenario 2 results* (NON-STATIONARY SYSTEM).** The results are presented in Table V. While all methods incur an overall higher cost in this scenario due to the non-stationarity of the system, we find that our method adapts effectively to system changes and obtains the best results alongside C-POMCP [43]. However, our method is more general than C-POMCP (which is customized for CAGE-2) and requires less online compute time. The policy adaptation of our method takes around 15 seconds with our computing hardware, whereas the methods based on deep RL require minutes or hours of optimization to adapt; see Fig. 21. Unlike the first scenario, increasing the rollout and lookahead horizons (m and ℓ) in Eq. (15) substantially improves performance at the expense of increased computational effort; see the third column in Table V.

Takeaway from the CAGE-2 evaluation.

Our method achieves performance on par with the state-of-the-art on the CAGE-2 benchmark while being more computationally efficient and adaptive.

VII. DISCUSSION OF THE EVALUATION RESULTS

The experimental evaluation highlights a key limitation of methods proposed in the research literature for computing security policies: they lack adaptability. In both the testbed and CAGE-2 evaluations, we find that most existing methods require lengthy retraining to adapt the policy to changes. This slow adaptation makes them unsuitable for modern networked systems, where configurations and workloads change frequently. Our method addresses this limitation by reducing the adaptation time, as shown in Fig. 18 and Fig. 21.

Another notable finding from our experiments is that while some of the current methods (e.g., deep RL methods) can yield effective security policies for stationary systems, they suffer from high variance and instability. For instance, in the testbed evaluation, we found the variance of PPO to be 10 times higher than that of our method. This variability can be explained by the tendency of PPO to converge to different local optima depending on the random seed. Indeed, in our experiments, we often needed to restart PPO several times with different random seeds to discover an effective policy. Such sensitivity to the random seed poses a potential operational concern, as it may lead to inconsistent policy performance across deployments. By contrast, the offline computation of our method converges reliably in all cases and provides performance guarantees; see Props. 1-2.

TABLE VI
HYPERPARAMETERS

Parameter(s)	*Values*
Convergence threshold of VI	0.1.
L [Eq. (16)]	20.
M, α	50, 0.99 .
Figure 9	$\text{BetaBin}(7, 1, 0.7)$ when $i^l = 1$. $\text{BetaBin}(7, 0.7, 3)$ when $i^l = 0$.

TABLE VII
NOTATION

Notation(s)	*Description*
$X, U, Z, B, \mathcal{F}$	State, control, observation, belief, and feature spaces; cf. §IV and §V-B.
$Q, \tilde{Q}$	Feature belief and representative feature belief spaces; cf. Eq. (8).
α, ρ	Discount factor and discretization resolution; cf. Eq. (8).
$q, \tilde{q}$	Feature belief and representative feature belief; cf. Eq. (8).
$(i, j), (x, y), n$	States, feature states, number of states; cf. §IV and §V-B.
b, u, z	Belief state, control, and observation; cf. §IV.
b_k, u_k, z_k, i_k	Belief state, control, observation, and state at time k; cf. §IV.
F, g, p	Belief estimator, stage cost, and observation distribution; cf. §IV.
$p_{ij}(u)$	Transition probabilities under control u; cf. §IV.
$\hat{b}$	Belief state estimated through the particle filter; cf. Eq. (7).
d_{yi}, ϕ_{jy}	Disaggregation probabilities and aggregation probabilities; cf. §V-B.
Φ	Aggregation mapping $\Phi : B \mapsto \tilde{Q}$; cf. Eq. (12).
$\hat{g}, \hat{p}$	Expected stage cost and observation probability given b; cf. Eqs. (3)–(5).
$J^\star, \mu^\star$	Optimal cost function and optimal policy; cf. Eq. (4).
J_μ	Cost function of policy μ; cf. Eq. (4).
$\tilde{J}$	Cost function approximation; cf. Eq. (13).
$r^\star, \pi^\star$	Optimal cost function and policy in the aggregate MDP; cf. Eq. (13).
$\mu, \tilde{\mu}$	Base policy [cf. Eq. (13)] and rollout policy [cf. Eq. (15)].
ℓ, m, L	Lookahead and rollout horizons, and number of simulations; cf. Eq. (15).
$\|\cdot\|, \Re, E\{\cdot\}$	The maximum norm, the real numbers, and the expectation operator.
$J_\mu, \tilde{J}_\mu$	Cost function of policy μ and estimated cost function of μ; cf. Eq. (16).
$S_{\tilde{q}}$	Belief space partition of the representative feature belief $\tilde{q}$; cf. Eq. (14).
ϵ	Maximum variation of $J^\star$ within a partition $S_{\tilde{q}}$; cf. Prop. 1.
K	Number of service replicas in the running example; cf. §III.
$A(\mu)$	The adaptation-completion metric of policy μ; cf. Eq. (18).

TABLE VIII
ATTACKER ACTIONS IN OUR TESTBED; ACTIONS ARE IDENTIFIED BY THE CORRESPONDING CVES [66] AND CWES [67]; THE ACTIONS ARE ALSO LINKED TO THE CORRESPONDING ATTACK TECHNIQUES IN MITRE ATT&CK [75]

Type	*Actions*	*MITRE ATT&CK technique*
Reconnaissance	TCP SYN scan, UDP scan	T1046 service scanning
	TCP XMAS scan	T1046 service scanning
	Vulscan	T1595 active scanning
	ping-scan	T1018 system discovery
Brute-force	Telnet, SSH	T1110 brute force
	FTP, Cassandra	T1110 brute force
	IRC, MongoDB, MySQL	T1110 brute force
	SMTP, PostgreSQL	T1110 brute force
Exploit	CVE-2017-7494	T1210 service exploitation
	CVE-2015-3306	T1210 service exploitation
	CVE-2010-0426	T1068 privilege escalation
	CVE-2015-5602	T1068 privilege escalation
	CVE-2015-1427	T1210 service exploitation
	CVE-2014-6271	T1210 service exploitation
	CVE-2016-10033	T1210 service exploitation
	SQLJ injection (CWE-89)	T1210 service exploitation

The method that comes closest in performance to our method on the CAGE-2 benchmark is C-POMCP [43, Alg. 1]. However, this method is tailored for CAGE-2 and not generalizable. For example, it cannot be directly applied to our testbed. Furthermore, our method requires less online computation and offers stronger theoretical guarantees.

Limitations. Our method has three main limitations. First, it requires a model or simulator of the target system to adapt the policy via rollout. As a result, its effectiveness depends on the accuracy with which this model captures the system dynamics. If the model is significantly misspecified, the quality of the adapted policy may deteriorate, as discussed in § V-C. Second, the performance of our method depends on the design of the feature space used for aggregation. Although effective features can often be selected based on engineering intuition, poor feature design can lead to degraded policy performance. Third, our method assumes a static attacker model, and extending it to account for a dynamic attacker that adapts its strategy in response to the security policy remains an open challenge.

VIII. CONCLUSION

Frequent adaptations of security policies are needed to keep pace with evolving security threats in networked systems. While RL is a promising approach to automate these adaptations, most methods proposed in the research literature lack performance guarantees and adapt slowly. Moreover, they have not been validated outside of simulation. This paper addresses these limitations by presenting and validating a scalable method for computing adaptive security policies with performance guarantees. It assumes a model or simulator of the target system and is based on three core ideas: (1) using particle filtering to estimate a belief about the system's security state; (2) aggregating beliefs to enable scalable offline policy computation; and (3) using rollout for online policy adaptation.

We show both theoretically and experimentally that our method provides advantages over other methods proposed in the research literature. Unlike existing methods that lack performance guarantees, we derive a bound on the approximation error of our aggregation scheme; see Props. 1 and 2. Moreover, we establish conditions under which our rollout method efficiently adapts policies to system changes; see Prop. 3. Simulations show that our method obtains state-of-the-art performance on the CAGE-2 benchmark; see Tables IV and V. Additionally, testbed experiments demonstrate its practicality; see Table III. While further testing remains to be done, these results indicate that our method provides a step towards reliable and automated adaptation of security policies in networked systems.

Future work. Several directions for future work merit investigation. First, a formal and empirical analysis of our method's sensitivity to model misspecification would be valuable. Second, the identification procedure in § VI-B could be extended into a more principled approach for estimating and updating model parameters from data. Third, the feature design in our method currently relies on domain knowledge; developing automated methods for feature extraction is a promising direction for future work. Fourth, our current POMDP formulation assumes a stochastic but non-adversarial system. Extending our approach to account for adversaries that adapt to the security policy is a natural direction for future work and would require a game-theoretic formulation.

APPENDIX A
PROOF OF PROP. 2

It can be shown that the optimal cost function $J^\star : B \mapsto \Re$ is uniformly continuous; see e.g., [72, Prop. 2.1]. Fix an arbitrary scalar $\gamma > 0$. By uniform continuity, there exists a scalar $\delta > 0$ such that

$$\|b - b'\| < \delta \implies |J^\star(b) - J^\star(b')| < \gamma \tag{19}$$

for all $b, b' \in B$, where $\|\cdot\|$ denotes the maximum norm.

Since $X = \mathcal{F}$ by assumption, we have that $\tilde{Q}$ is a finite subset of B. Therefore, the discretization in Eq. (8) partitions B into grid cells $S_{\tilde{q}}$ with resolution $\rho \geq 1$; cf. Eq. (14). Further, Eq. (10) implies that if $b \in S_{\tilde{q}}$, then

$$\|b - \tilde{q}\| = \min_{\tilde{q}' \in \tilde{Q}} \|b - \tilde{q}'\|.$$

Because each belief coordinate $b(i)$ lies in $[0, 1]$ and each representative feature belief coordinate $\tilde{q}(i)$ equals $\frac{\beta_i}{\rho}$ for some $\beta_i \in \{0, \ldots, \rho\}$ [cf. Eq. (8)], we have

$$\max_{b,b' \in S_{\tilde{q}}} \|b - b'\| \leq \frac{2n}{\rho}, \qquad \text{for every } \tilde{q} \in \tilde{Q}.$$

Choose any ρ such that $\frac{2n}{\rho} < \delta$. By Eq. (19), we have

$$|J^\star(b) - J^\star(b')| < \gamma, \quad \text{for all } b, b' \in S_{\tilde{q}},\ \tilde{q} \in \tilde{Q}.$$

Because $\gamma > 0$ is arbitrary and there exists a large enough ρ such that $\frac{n}{\rho} < \delta$ for any $\delta > 0$, we have

$$\lim_{\rho \to \infty} \max_{\tilde{q} \in \tilde{Q}} \max_{b,b' \in S_{\tilde{q}}} |J^\star(b) - J^\star(b')| = 0.$$

Hence the constant ϵ in Prop. 1 diminishes as $\rho \to \infty$. Invoking the error bound in Prop. 1 completes the proof. □

Appendix B
Experimental Setup

All computations are performed on an M2 Ultra processor. The attacker actions in our testbed are listed in Table VIII. The hyperparameters are listed in Table VI. Notation is explained in Table VII. We use the implementation of Cardiff described in [29] and the implementation of C-POMCP described in [43]. For PPO and DQN, we use the STABLE-BASELINES implementations [73]. For PPG, we use the CLEAN-RL implementation [74]. For POMCP, we use our implementation [62]. We set the hyperparameters for these methods to be the same as those used in [43]. Unless stated otherwise, we run PPO and PPG with a vector of the sample states of the particle filter as input. We identify the dynamics of the aggregate MDP in Fig. 6 through simulations of the original POMDP. We solve the aggregate MDP using value iteration (VI) with a convergence tolerance of $\epsilon = 10^{-6}$. Hyperparameters and feature selections are chosen based on domain knowledge and limited experimentation (1–3 trials), without extensive tuning. Reported results for all methods are averaged over multiple independent runs with different random seeds; unless otherwise specified, each method is run 5 times.

Appendix C
Testbed Setup

The network topology of the networked system that we run on our testbed is shown in Fig. 4 and the configuration is listed in Table II. Hosts and switches are emulated with Docker containers. Resource allocation to containers is enforced using cGroups. Network connectivity between containers is emulated with virtual links implemented by Linux bridges and network namespaces, which create logical copies of the physical host's network stack. Network conditions of virtual links are created using the NetEm module in the Linux kernel. We emulate connections between servers with full-duplex lossless connections of 1 Gbit/s capacity in both directions. Similarly, we emulate connections between servers and clients with full-duplex connections of 100 Mbit/s capacity and 0.1% packet loss with random bursts of 1% packet loss. These numbers are based on measurements on enterprise networks [76].

The client population is emulated through processes that access services on emulated hosts. Client arrivals are controlled by a Poisson process with exponentially distributed service times. The sequence of service invocations is selected uniformly at random. Similarly, the attacker is emulated by programs that select actions from the list in Table VIII. The source code of our emulation platform is available at [62].

References

[1] D. E. Denning, "An intrusion-detection model," *IEEE Trans. Softw. Eng.*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.

[2] V. Varadharajan, K. Karmakar, U. Tupakula, and M. Hitchens, "A policy-based security architecture for software-defined networks," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 4, pp. 897–912, Apr. 2019.

[3] T. T. Nguyen and V. J. Reddi, "Deep reinforcement learning for cyber security," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 8, pp. 3779–3795, Aug. 2023.

[4] T. Avgerinos et al., "The mayhem cyber reasoning system," *IEEE Secur. Privacy*, vol. 16, no. 2, pp. 52–60, Mar. 2018.

[5] S. A. Zonouz, H. Khurana, W. H. Sanders, and T. M. Yardley, "RRE: A game-theoretic intrusion response and recovery engine," in *Proc. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Jun. 2009, pp. 439–448.

[6] G. Deng et al., "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Secur. Symp. (USENIX Secur.)*, Philadelphia, PA, USA, Aug. 2024, pp. 847–864.

[7] T. Zhang et al., "When moving target defense meets attack prediction in digital twins: A convolutional and hierarchical reinforcement learning approach," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 10, pp. 3293–3305, Oct. 2023.

[8] K. Hammar and R. Stadler, "Intrusion tolerance for networked systems through two-level feedback control," in *Proc. 54th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, Jun. 2024, pp. 338–352.

[9] D. Wang, Z. Zhang, H. Zhang, Z. Qian, S. V. Krishnamurthy, and N. Abu-Ghazaleh, "SyzVegas: Beating kernel fuzzing odds with reinforcement learning," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, Aug. 2021, pp. 2741–2758.

[10] D. Bertsekas, *Dynamic Programming and Optimal Control*, vol. 2, 4th ed. Belmont, MD, USA: Athena Scientific, 2012.

[11] D. Bertsekas, *Rollout, Policy Iteration, and Distributed Reinforcement Learning*. Belmont, MD, USA: Athena Scientific, 2021.

[12] G. Tesauro and G. Galperin, "On-line policy improvement using Monte-Carlo search," in *Advances in Neural Information Processing Systems*, vol. 9, M. Mozer, M. Jordan, and T. Petsche, Eds., Cambridge, MA, USA: MIT Press, 1996.

[13] H. Yu and D. Bertsekas, "Discretized approximations for POMDP with average cost," in *Proc. 20th Conf. Uncertainty Artif. Intell.*, Arlington, VA, USA, 2004, pp. 619–627.

[14] N. Saldi, S. Yüksel, and T. Linder, "On the asymptotic optimality of finite approximations to Markov decision processes with borel spaces," *Math. Oper. Res.*, vol. 42, no. 4, pp. 945–978, Nov. 2017.

[15] D. P. Bertsekas, "Feature-based aggregation and deep reinforcement learning: A survey and some new implementations," *IEEE/CAA J. Automatica Sinica*, vol. 6, no. 1, pp. 1–31, Jan. 2019.

[16] D. Bertsekas, *A Course in Reinforcement Learning*, 2nd ed. Belmont, MD, USA: Athena Scientific, 2025.

[17] D. Bertsekas, "Biased aggregation, rollout, and enhanced policy improvement for reinforcement learning," 2019, *arXiv:1910.02426*.

[18] F. S. Samani, K. Hammar, and R. Stadler, "Online policy adaptation for networked systems using rollout," in *Proc. IEEE Netw. Oper. Manage. Symp.*, May 2024, pp. 1–9.

[19] H. Liu, Y. Li, J. Mårtensson, L. Xie, and K. H. Johansson, "Reinforcement learning based approach for flip attack detection," in *Proc. 59th IEEE Conf. Decis. Control (CDC)*, Dec. 2020, pp. 3212–3217.

[20] H. Liu, Y. Li, K. H. Johansson, J. Mårtensson, and L. Xie, "Rollout approach to sensor scheduling for remote state estimation under integrity attack," *Automatica*, vol. 144, Oct. 2022, Art. no. 110473.

[21] K. Hammar, T. Li, R. Stadler, and Q. Zhu, "Adaptive security response strategies through conjectural online learning," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 4055–4070, 2025.

[22] T. Bai, Y. Li, K. H. Johansson, and J. Mårtensson, "Rollout-based charging strategy for electric trucks with hours-of-service regulations," *IEEE Control Syst. Lett.*, vol. 7, pp. 2167–2172, 2023.

[23] D. P. Bertsekas and D. A. Castanon, "Adaptive aggregation methods for infinite horizon dynamic programming," *IEEE Trans. Autom. Control*, vol. 34, no. 6, pp. 589–598, Jun. 1989.

[24] K. Hammar and T. Alpcan, "Scalable solutions to zero-sum partially observable stochastic games through belief aggregation with approximation guarantees," in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2026, vol. 40, no. 20, pp. 17023–17031.

[25] T. Alpcan and T. Basar, *Network Security: A Decision and Game-Theoretic Approach*, 1st ed. New York, NY, USA: Cambridge Univ. Press, 2010.

[26] K. Hammar, "Optimal security response to network intrusions in IT systems," Ph.D. dissertation, KTH Roy. Inst. Technol., Stockholm, Sweden, 2024.

[27] S. Vyas, V. Mavroudis, and P. Burnap, "Towards the deployment of realistic autonomous cyber network defence: A systematic review," *ACM Comput. Surv.*, vol. 58, no. 1, pp. 1–36, Jan. 2026.

[28] M. Kiely, D. Bowman, M. Standen, and C. Moir, "On autonomous agents in a cyber defence environment," 2023, *arXiv:2309.07388*.

[29] S. Vyas, J. Hannay, A. Bolton, and P. Pete Burnap, "Automated cyber defence: A review," 2023, *arXiv:2303.04926*.

[30] E. Bates, V. Mavroudis, and C. Hicks, "Reward shaping for happier autonomous cyber security agents," in *Proc. 16th ACM Workshop Artif. Intell. Secur.*, New York, NY, USA, Nov. 2023, pp. 221–232.

[31] M. Wolk et al., "Beyond CAGE: Investigating generalization of learned autonomous network defense policies," 2022, *arXiv:2211.15557*.

[32] M. Foley, C. Hicks, K. Highnam, and V. Mavroudis, "Autonomous network defence using reinforcement learning," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, New York, NY, USA, May 2022, pp. 1252–1254.

[33] M. Foley, M. Wang, Z. M, C. Hicks, and V. Mavroudis, "Inroads into autonomous network defence using explained reinforcement learning," 2023, *arXiv:2306.09318*.

[34] S. Xu, Z. Xie, C. Zhu, X. Wang, and L. Shi, "Enhancing cybersecurity in industrial control system with autonomous defense using normalized proximal policy optimization model," in *Proc. IEEE 29th Int. Conf. Parallel Distrib. Syst. (ICPADS)*, Dec. 2023, pp. 928–935.

[35] Z. Cheng, X. Wu, J. Yu, S. Yang, G. Wang, and X. Xing, "RICE: Breaking through the training bottlenecks of reinforcement learning with explanation," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 1–26.

[36] G. Palmer et al., "An empirical game-theoretic analysis of autonomous cyber-defence agents," 2025, *arXiv:2501.19206*.

[37] K. Heckel, "Neuroevolution for autonomous cyber defense," in *Proc. Companion Conf. Genetic Evol. Comput.*, New York, NY, USA, Jul. 2023, pp. 651–654.

[38] Y. Tang, J. Sun, H. Wang, J. Deng, L. Tong, and W. Xu, "A method of network attack-defense game and collaborative defense decision-making based on hierarchical multi-agent reinforcement learning," *Comput. Secur.*, vol. 142, Jul. 2024, Art. no. 103871.

[39] J. Wiebe, R. Al Mallah, and L. Li, "Learning cyber defence tactics from scratch with multi-agent reinforcement learning," 2023, *arXiv:2310.05939*.

[40] A. Vikram Singh et al., "Hierarchical multi-agent reinforcement learning for cyber network defense," 2024, *arXiv:2410.17351*.

[41] Y. Yan, Y. Zhang, and K. Huang, "Depending on yourself when you should: Mentoring LLM with RL agents to become the master in cybersecurity games," 2024, *arXiv:2403.17674*.

[42] J. F. Loevenich, E. Adler, R. Mercier, A. Velazquez, and R. R. F. Lopes, "Design of an autonomous cyber defence agent using hybrid AI models," in *Proc. Int. Conf. Mil. Commun. Inf. Syst. (ICMCIS)*, Apr. 2024, pp. 1–10.

[43] K. Hammar, N. Dhir, and R. Stadler, "Optimal defender strategies for CAGE-2 using causal modeling and tree search," 2024, *arXiv:2407.11070*.

[44] A. Ramamurthy and N. Dhir, "General autonomous cybersecurity defense: Learning robust policies for dynamic topologies and diverse attackers," 2025, *arXiv:2506.22706*.

[45] H. Mohammadi, J. J. Davis, and M. Kiely, "Leveraging large language models for autonomous cyber defense: Insights from CAGE-2 simulations," *IEEE Intell. Syst.*, vol. 40, no. 4, pp. 29–36, Jul. 2025.

[46] Atlassian and C. Research. (2020). *2020 DevOps Trends Survey*. [Online]. Available: https://www.atlassian.com/whitepapers/devops-survey-2020

[47] New Relic and Enterprise Technology Research (ETR), *2024 Observability Forecast Report*, San Francisco, CA, USA, 2024.

[48] A. Aydeger, M. H. Manshaei, M. A. Rahman, and K. Akkaya, "Strategic defense against stealthy link flooding attacks: A signaling game approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 1, pp. 751–764, Jan. 2021.

[49] Q. Zhu and T. Başar, "Game-theoretic approach to feedback-driven multi-stage moving target defense," in *Decision and Game Theory for Security*, S. K. Das, C. Nita-Rotaru, and M. Kantarcioglu, Eds., Cham, Switzerland: Springer, 2013, pp. 246–263.

[50] K. Hammar and R. Stadler, "Intrusion prevention through optimal stopping," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 3, pp. 2333–2348, Sep. 2022.

[51] E. Miehling, M. Rasouli, and D. Teneketzis, "A POMDP approach to the dynamic defense of large-scale cyber networks," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 10, pp. 2490–2505, Oct. 2018.

[52] Y. Yue, S. Tan, Y. Tao, N. Liu, and J. Lü, "A non-Markovian game approach on labeled attack graphs for security decision-making in industrial control systems," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 10511–10524, 2025.

[53] F. Borrelli, A. Bemporad, and M. Morari, *Predictive Control for Linear and Hybrid Systems*. Cambridge, U.K.: Cambridge Univ. Press, 2017.

[54] K. Åström and B. Wittenmark, *Adaptive Control*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1995.

[55] K. J. Åström, "Optimal control of Markov processes with incomplete state information," *J. Math. Anal. Appl.*, vol. 10, no. 1, pp. 174–205, Feb. 1965.

[56] V. Krishnamurthy, *Partially Observed Markov Decision Processes: From Filtering to Controlled Sensing*. Cambridge, U.K.: Cambridge Univ. Press, 2016.

[57] D. Crisan and A. Doucet, "A survey of convergence results on particle filtering methods for practitioners," *IEEE Trans. Signal Process.*, vol. 50, no. 3, pp. 736–746, Mar. 2002.

[58] Y. Li, K. Hammar, and D. Bertsekas, "Feature-based belief aggregation for partially observable Markov decision problems," 2025, *arXiv:2507.04646*.

[59] D. Bertsekas, *Reinforcement Learning and Optimal Control*. Belmont, MD, USA: Athena Scientific, 2019.

[60] L. Ljung, *System Identification*. Boston, MA, USA: Birkhäuser, 1998, pp. 163–173.

[61] G. N. Iyengar, "Robust dynamic programming," *Math. Oper. Res.*, vol. 30, no. 2, pp. 257–280, 2005.

[62] K. Hammar. (2025). *Software for the paper 'Adaptive Network Security Policies via Belief Aggregation and Rollout'*. o https://github.com/Limmen/rollout_aggregation and [Online]. Available: https://github.com/Limmen/csle

[63] A. Cassandra, M. L. Littman, and N. L. Zhang, "Incremental pruning: A simple, fast, exact method for partially observable Markov decision processes," in *Proc. 13th Conf. Uncertainty Artif. Intell.*, 1997, pp. 54–61.

[64] D. Bertsekas, *Lessons from AlphaZero for Optimal, Model Predictive, and Adaptive Control*. Belmont, MD, USA: Athena Scientific, 2022.

[65] J. MacQueen et al., "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Probab.*, Oakland, CA, USA, vol. 1, 1967, pp. 281–297.

[66] The MITRE Corporation. (2022). *CVE Database*. [Online]. Available: https://cve.mitre.org/

[67] (2023). *CWE List*. [Online]. Available: https://cwe.mitre.org/index.html

[68] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.

[69] K. W. Cobbe, J. Hilton, O. Klimov, and J. Schulman, "Phasic policy gradient," in *Proc. 38th Int. Conf. Mach. Learn.*, vol. 139, Jul. 2021, pp. 2020–2027.

[70] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.

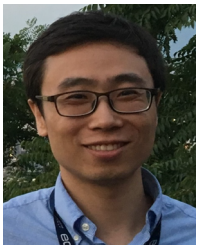
[71] D. Silver and J. Veness, "Monte-Carlo planning in large POMDPs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 23, 2010, pp. 1–9.

[72] H. Yu, "Approximate solution methods for partially observable Markov and semi-Markov decision processes," Ph.D. dissertation, Massachusetts Inst. Technol., Cambridge, MA, USA, 2006.

[73] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.
[74] S. Huang et al., "CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms," *J. Mach. Learn. Res.*, vol. 23, p. 274, Jan. 2022.
[75] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," MITRE Corp., McLean, VA, USA, Tech. Rep. MP180360R1, 2018.
[76] V. Paxson, "End-to-end internet packet dynamics," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 27, no. 4, pp. 139–152, Oct. 1997.



Kim Hammar received the M.Sc. and Ph.D. degrees in electrical engineering from the KTH Royal Institute of Technology, Sweden, in 2018 and 2024, respectively. He is currently a Post-Doctoral Researcher with joint affiliations at KTH, Arizona State University, USA, The University of Melbourne, Australia, and Imperial College London, U.K. His main research interests are automatic control and management of networked systems. He was a recipient of the VR Post-Doctoral Fellowship 2025. He is an Associate Editor of IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT.



Yuchao Li received the B.Eng. degree in mechanical engineering from Harbin Institute of Technology, Harbin, China, in 2015, and the M.Sc. degree in mechatronics and the Ph.D. degree in electrical engineering from the Royal Institute of Technology, Stockholm, Sweden, in 2017 and 2023, respectively.

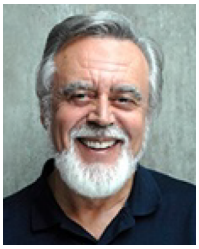
Since August 2023, he has been a Post-Doctoral Research Scholar with Arizona State University, Tempe, USA. His research interests include optimal control, reinforcement learning, network optimization, and their applications to transportation and server systems.



Tansu Alpcan (Senior Member, IEEE) received the Ph.D. degree in electrical and computer engineering from the University of Illinois Urbana–Champaign (UIUC) in 2006. His research interests include applications of optimization and game theories and machine learning to security and resource allocation problems in communications, smart grids, and the Internet of Things. He has worked as a Senior Research Scientist with Deutsche Telekom Laboratories, Berlin, Germany (2006–2009) and as an Assistant Professor (Junior Professur) at Technical University Berlin (2009–2011). He is currently with the Department of Electrical and Electronic Engineering, The University of Melbourne, as a Professor and a Reader. He is the (co-)author of more than 200 journals and conference articles and the book *Network Security: A Decision and Game Theoretic Approach* [Cambridge University Press (CUP), 2011]. He co-edited the book *Mechanisms and Games for Dynamic Spectrum Allocation* (CUP, 2014). He chaired or was an associate editor, the TPC chair, or a TPC member of several prestigious IEEE workshops, conferences, and journals.



Emil C. Lupu (Member, IEEE) is currently a Professor in computer systems with the Department of Computing, Imperial College London, where he leads the Resilient Information Systems Security Group (https://www.rissgroup.org). He is also the Co-Director of the Research Institute in Trustworthy Inter-Connected Cyber-Physical Systems (https://www.ritics.org). His research focuses on the resilience and security of cyber-physical systems including large scale infrastructures and embedded and autonomous systems. His research contributions include new models for evaluating systems' resilience and the effect of security investment, approaches for the integration of security and safety methodologies, new algorithms for the detection of malicious data injections, and evaluating robustness of sensor infrastructures to adversarial attacks and new attacks and defenses against machine learning models and components in both centralized and distributed/federated settings.



Dimitri P. Bertsekas received the degree in engineering from the National Technical University of Athens, Greece, the M.S. degree in electrical engineering from George Washington University, Washington, DC, USA, in 1969, and the Ph.D. degree in system science from Massachusetts Institute of Technology (MIT), Cambridge, MA, USA, in 1971.

He has held faculty positions with the Engineering-Economic Systems Department, Stanford University (1971–1974) and the Electrical Engineering Department, University of Illinois Urbana–Champaign, Urbana (1974–1979). From 1979 to 2019, he was with the Electrical Engineering and Computer Science Department, MIT, where he worked as the McAfee Professor of engineering. Since 2019, he has been a Fulton Professor in computational decision making at the School of Computing and Augmented Intelligence, Arizona State University (ASU), Tempe. His recent books are *Reinforcement Learning and Optimal Control* (2019), *Rollout, Policy Iteration, and Distributed Reinforcement Learning* (2020), *Abstract Dynamic Programming* (2022, third edition), *Lessons from AlphaZero for Optimal, Model Predictive, and Adaptive Control* (2022), and *A Course in Reinforcement Learning* (2025, second edition), all published by Athena Scientific.

Dr. Bertsekas was awarded the INFORMS 1997 Prize for Research Excellence in the Interface Between Operations Research and Computer Science for his book "Neuro-Dynamic Programming," the 2001 ACC John R. Ragazzini Education Award, the 2009 INFORMS Expository Writing Award, the 2014 ACC Richard E. Bellman Control Heritage Award for "contributions to the foundations of deterministic and stochastic optimization-based methods in systems and control," the 2014 Khachiyan Prize for Life-Time Accomplishments in Optimization, the SIAM/MOS 2015 George B. Dantzig Prize, and the 2022 IEEE Control Systems Award. Together with his co-author John Tsitsiklis, he was awarded the 2018 INFORMS John von Neumann Theory Prize, for the contributions of the research monographs "Parallel and Distributed Computation" and "Neuro-Dynamic Programming." In 2001, he was elected to U.S. National Academy of Engineering for "pioneering contributions to fundamental research, practice and education of optimization/control theory."